Smart Parking Management System - Project Documentation

PROJECT CHARTER

Project Title

Smart Parking Management System with Computer Vision and Edge Computing

Project Start Date: November 4, 2024

Target Completion Date: December 8, 2024 (5 weeks)

Project Manager

[To be assigned from team]

Project Sponsor/Stakeholder

[Course Instructor/Department Name]

1. EXECUTIVE SUMMARY

The Smart Parking Management System is an innovative IoT and computer vision solution designed to automate parking space detection, provide real-time availability information, and enable intelligent parking recommendations. The system leverages YOLOv8 object detection, OAK-D depth cameras, Raspberry Pi edge computing, and AWS cloud infrastructure to create a seamless user experience that reduces parking search time and optimizes space utilization.

Accelerated Timeline: This project operates on an aggressive 5-week sprint schedule with weekly deliverables, requiring focused execution and parallel workstreams.

2. PROJECT OBJECTIVES

Primary Objectives:

1. Develop a real-time parking space detection system with >85% accuracy
2. Deploy edge computing solution on Raspberry Pi with OAK-D camera
3. Implement spatial analysis to calculate distances from entrance
4. Create cloud infrastructure for data processing and storage
5. Build user-facing web application for parking availability visualization
6. Achieve <3 second latency from detection to display

7. Implement automated billing based on parking duration

Success Criteria:

- System accurately detects parking space occupancy in real-time
- "Nearest spot" algorithm correctly identifies optimal parking
- End-to-end latency remains under 3 seconds
- System handles minimum 20 parking spaces simultaneously
- Frontend displays accurate, real-time data
- Billing calculations are accurate to the minute
- Complete technical documentation delivered
- Successful live demonstration to stakeholders by December 8th

3. PROJECT SCOPE

In Scope:

Hardware & Infrastructure

- Raspberry Pi setup with OAK-D camera integration
- Edge deployment on single Raspberry Pi device
- AWS cloud infrastructure (Lambda/EC2, DynamoDB, API Gateway)
- Web hosting for frontend application

Software Development

- Custom parking dataset collection and annotation (100+ images)
- YOLOv8 Nano model training for parking detection
- Model conversion to Myriad X format (.blob)
- RGB-Depth alignment pipeline
- 2D to 3D spatial coordinate transformation
- Nearest parking spot sorting algorithm
- Edge-to-cloud communication (MQTT/HTTP)
- RESTful API for data management
- Web-based frontend (entry screen + dashboard)
- Automated billing system

Features

- Real-time parking occupancy detection
- Distance calculation relative to entrance
- Nearest available spot recommendations

Smart Parking Management System - Project Documentation

- Live dashboard with parking lot visualization
- Entry/exit time tracking
- Duration-based billing calculation
- Admin reporting capabilities

Documentation

- Technical architecture documentation
- Spatial analysis methodology report
- Cloud infrastructure report
- User manual
- Deployment guide
- API documentation
- Final presentation materials

Out of Scope:

Explicitly Excluded:

- Mobile native applications (iOS/Android)
- Multiple camera/location deployment
- License plate recognition (LPR/ANPR)
- Payment gateway integration (Stripe, PayPal, etc.)
- Vehicle type classification beyond basic detection
- Reserved parking or advance booking system
- Integration with existing parking management systems
- Physical barrier gate control systems
- SMS/email notification system
- Multi-tenant/multi-location management
- Parking violation detection
- Historical analytics and reporting dashboard
- Machine learning model retraining automation
- Load balancing for high-traffic scenarios

Future Enhancements (Post-Project):

- Scalability to multiple parking lots
- Advanced analytics and predictive availability
- Integration with navigation apps (Google Maps, Waze)
- Dynamic pricing based on demand
- Permit and subscription management
- Integration with smart city infrastructure

Smart Parking Management System - Project Documentation

4. DELIVERABLES

Sprint 1: Setup & Foundation *(Week 1: Nov 4-10)*

- ☐ Raspberry Pi with OS and OAK-D dependencies installed
- ☐ GitHub repository with branch structure
- ☐ Annotated parking dataset (100+ images) in YOLO format
- ☐ Initial YOLOv8 training environment setup

Sprint 2: Model Development & Spatial Pipeline *(Week 2: Nov 11-17)*

- ☐ Trained YOLOv8 Nano model with validation metrics
- ☐ Performance evaluation report (confusion matrix, mAP scores)
- ☐ Converted .blob model for OAK-D deployment
- ☐ OAK-D RGB-Depth alignment pipeline

Sprint 3: Edge & Cloud Infrastructure *(Week 3: Nov 18-24)*

- ☐ 2D-to-3D coordinate transformation module
- ☐ Nearest spot sorting algorithm implementation
- ☐ main.py with complete edge processing logic
- ☐ AWS backend infrastructure (Lambda/EC2 + DynamoDB)
- ☐ MQTT/HTTP cloud communication module

Sprint 4: Frontend & Billing *(Week 4: Nov 25-Dec 1)*

- ☐ RESTful API endpoints with documentation
- ☐ Web frontend (entry screen and dashboard)
- ☐ Billing system implementation
- ☐ Frontend-backend integration

Sprint 5: Integration, Optimization & Documentation *(Week 5: Dec 2-8)*

- ☐ End-to-end system integration
- ☐ Performance optimization (<3 sec latency)
- ☐ Comprehensive testing report
- ☐ Final technical documentation (Spatial + Cloud reports)
- ☐ User manual and deployment guide
- ☐ Project presentation and live demo

5. PROJECT ORGANIZATION

Smart Parking Management System - Project Documentation

Team Structure *(3 Members)*

Team Member: Naimul

Role: ML/Hardware Lead **Responsibilities:**

- YOLOv8 model training and optimization
- Model conversion to .blob format
- Model performance evaluation and tuning
- Raspberry Pi and OAK-D hardware setup
- Hardware-software integration testing
- Model deployment on edge device

Key Deliverables:

- Trained YOLOv8 model
- .blob model file
- Performance metrics report
- Raspberry Pi setup documentation

Team Member: Armaan

Role: Data & Cloud Lead **Responsibilities:**

- Dataset collection and annotation (100+ images)
- Data quality assurance and validation
- AWS backend architecture and implementation
- DynamoDB schema design
- API Gateway and Lambda functions
- Cloud deployment and configuration
- Data pipeline from edge to cloud

Key Deliverables:

- Annotated parking dataset
- AWS cloud infrastructure
- RESTful API
- Cloud infrastructure report
- Data flow documentation

Team Member: Honor Byansi

Role: Frontend Design & Documentation Lead **Responsibilities:**

Smart Parking Management System - Project Documentation

- Web application UI/UX design
- Frontend development (entry screen + dashboard)
- Real-time data visualization
- Billing system interface
- User manual creation
- Technical documentation writing
- Deployment guide preparation
- Final presentation materials

Key Deliverables:

- Web frontend application
- User interface designs
- User manual
- Deployment guide
- Spatial analysis report
- Final presentation

Joint Responsibility: Optimization & Integration

All Team Members Responsibilities:

- System integration testing
- End-to-end workflow validation
- Performance optimization
- Latency reduction (<3 sec)
- Bug fixing and troubleshooting
- Code review and quality assurance
- Final system validation

Key Deliverables:

- Integrated system
- Performance optimization report
- Test results documentation
- Final working demo

RACI Matrix

Task/Deliverable	Naimul	Armaan	Honor	Joint
Hardware Setup	**R**	C	I	-

Smart Parking Management System - Project Documentation

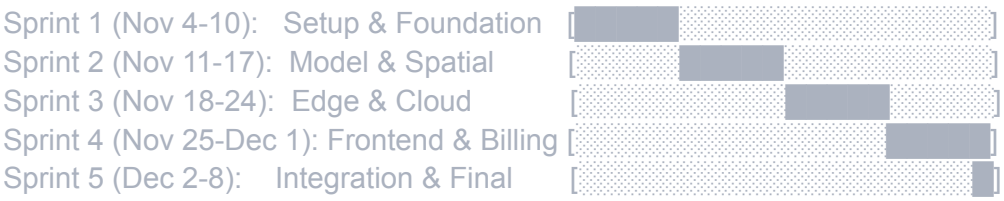
GitHub Repository	C	**R**	C	-
Dataset Collection	C	**R**	I	-
Dataset Annotation	C	**R**	C	-
Model Training	**R**	I	I	-
Model Evaluation	**R**	C	I	-
Model Conversion	**R**	I	I	-
Spatial Analysis	C	**R**	I	-
Edge Pipeline	**R**	C	I	-
Cloud Backend	I	**R**	I	-
API Development	I	**R**	C	-
Frontend Design	I	C	**R**	-
Frontend Development	I	C	**R**	-
Billing System	I	**R**	C	-
System Integration	C	C	C	**R**
Optimization	C	C	C	**R**
Testing	C	C	C	**R**
User Manual	I	I	**R**	-
Technical Docs	C	C	**R**	-
Presentation	C	C	**R**	-

Legend: R = Responsible, A = Accountable, C = Consulted, I = Informed

6. PROJECT TIMELINE & MILESTONES

Smart Parking Management System - Project Documentation

5-Week Sprint Overview



Critical Milestones

Milestone	Target Date	Dependencies	Deliverables	Owner
M1: Foundation Ready	Nov 10 (Sun)	None	Hardware setup, Dataset ready, Training started	Naimul, Armaan
M2: Model & Pipeline Complete	Nov 17 (Sun)	M1	Trained model, .blob file, Spatial pipeline	Naimul, Armaan
M3: Infrastructure Live	Nov 24 (Sun)	M2	Edge device working, Cloud backend, API live	Naimul, Armaan
M4: User Interface Ready	Dec 1 (Sun)	M3	Frontend deployed, Billing system, End-to-end flow	Honor, Armaan
M5: Project Complete	Dec 8 (Sun)	M4	Optimized system, Documentation, Final demo	All

8. RESOURCE REQUIREMENTS

Hardware Resources

Item	Quantity	Cost (Est.)	Owner	Status
Raspberry Pi 4 (4GB+)	1	$55	Team	Required Week 1
OAK-D Camera	1	$150	Team/School	Required Week 1

Smart Parking Management System - Project Documentation

MicroSD Card (32GB+)	1	$10	Team	Required Week 1
Power Supply	1	$10	Team	Required Week 1
USB-C Cable	1	$5	Team	Required Week 1
TOTAL		**$230**		

Hardware Procurement Timeline:

- **By Nov 4:** Order all hardware (if purchasing)
- **By Nov 6:** All hardware received and ready for setup

Software & Cloud Resources

Resource	Type	Cost	Purpose	Setup By
AWS Free Tier	Cloud	Free	Lambda, DynamoDB, API Gateway	Nov 18
GitHub	Repository	Free	Version control	Nov 4
Python 3.8+	Language	Free	Development	Nov 4
YOLOv8 (Ultralytics)	ML Framework	Free	Object detection	Nov 4
DepthAI	Library	Free	OAK-D integration	Nov 5
React/Vue	Frontend	Free	Web application	Nov 11
Roboflow/LabelImg	Annotation	Free	Dataset labeling	Nov 5

Software Setup Timeline:

- **Week 1:** All development environments configured
- **Week 2:** ML training environment ready
- **Week 3:** Cloud infrastructure live

Smart Parking Management System - Project Documentation

Human Resources

Time Commitment per Team Member:

- **Expected:** 20 hours/week per person
- **Total Team:** 60 hours/week
- **Project Duration:** 5 weeks
- **Total Project Hours:** 300 person-hours

Time Allocation by Sprint:

Sprint	Focus Area	Hours/Sprint	Cumulative
Sprint 1	Setup & Foundation	60 hours	60
Sprint 2	Model & Pipeline	60 hours	120
Sprint 3	Infrastructure	60 hours	180
Sprint 4	Features & UI	60 hours	240
Sprint 5	Integration & Docs	60 hours	300

Weekly Time Breakdown per Person:

- Development: 12 hours
- Testing: 3 hours
- Meetings: 3 hours
- Documentation: 2 hours
- **Total:** 20 hours/week

9. RISK MANAGEMENT

Risk Register

Risk ID	Risk Description	Prob.	Impact	Risk Score	Mitigation Strategy	Owner
R1	Hardware arrives late or DOA	Medium	Critical	**HIGH**	Order by Nov 4; have backup plan using laptop webcam + simulated depth	Naimul

Smart Parking Management System - Project Documentation

R2	Model accuracy <85%	Medium	High	**HIGH**	Start training early; use pre-trained weights; augment data	Naimul
R3	Latency exceeds 3 seconds	High	High	**CRITICAL**	Profile early Week 2; optimize incrementally; accept 4-5 sec if necessary	All
R4	AWS costs exceed free tier	Low	Medium	**LOW**	Set billing alerts at $5, $10; monitor daily	Armaan
R5	Dataset quality insufficient	Medium	High	**HIGH**	Begin collection Nov 4; ensure 100+ by Nov 8; use augmentation	Armaan
R6	Team member sick/unavailable	Medium	Medium	**MEDIUM**	Cross-document all work; pair programming; clear handoff docs	All
R7	Depth accuracy poor	Medium	High	**HIGH**	Calibrate early; test Week 2; fallback to 2D-only if needed	Naimul
R8	Integration problems	High	High	**CRITICAL**	Define API contracts early; weekly integration tests; buffer time Sprint 5	All
R9	.blob conversion fails	Low	High	**MEDIUM**	Test conversion Week 2; use proven tools; fallback to cloud inference	Naimul
R10	Scope creep	Medium	Medium	**MEDIUM**	Strict scope adherence; defer enhancements; track "future work"	All
R11	Thanksgiving disruption	High	Low	**MEDIUM**	Front-load critical work; flexible Wed-Thu; no critical demos	All

Smart Parking Management System - Project Documentation

R12	Demo day technical failure	Medium	High	**HIGH**	Practice demo 3x; have backup video; test environment early	All
R13	Time underestimation	High	High	**CRITICAL**	5-week timeline is aggressive; work weekends if needed; prioritize ruthlessly	All

Critical Risk Mitigation Plans

R3: Latency Exceeds Target (CRITICAL)

Why Critical: Core requirement; affects user experience

Prevention:

- Week 2: Measure baseline latency for each component
- Week 3: Set latency budget (inference 1s, network 0.5s, cloud 0.5s, frontend 0.5s)
- Week 4: Continuous monitoring and optimization

Mitigation if it occurs:

- Accept 4-5 second latency as "acceptable" vs ideal
- Reduce camera FPS from 30 to 15 if needed
- Process every other frame
- Implement client-side prediction

Escalation:

- If >5 seconds by Dec 3: Simplify to single-shot detection (no continuous streaming)

R8: Integration Problems (CRITICAL)

Why Critical: Multiple components must work together; little time buffer

Prevention:

- Week 1: Define all interface contracts (JSON schemas, API endpoints)
- Week 2: Create integration test framework
- Week 3: Daily integration tests

- Week 4: Continuous integration

Mitigation if it occurs:

- Implement mocking layers for isolated testing
- Use feature flags to disable problematic components
- Pair programming between component owners
- Extend Sprint 5 into weekend if needed

Escalation:

- If major issues by Dec 5: Simplify to working subset (e.g., remove billing)

R13: Time Underestimation (CRITICAL)

Why Critical: 5 weeks is very aggressive for this scope

Prevention:

- Ruthless prioritization: MVP first, enhancements never
- Work weekends if falling behind
- Daily standup to catch delays early
- Cut scope aggressively if needed

Mitigation if it occurs:

- Identify must-have vs nice-to-have features by Week 3
- Defer documentation polish to last 2 days
- Accept "working" over "perfect"
- Team all-hands on critical blockers

Escalation:

- If significantly behind by Nov 24: Negotiate scope reduction with instructor

10. COMMUNICATION PLAN

Team Meetings

Smart Parking Management System - Project Documentation

Meeting	Day/Time	Duration	Attendees	Purpose	Format
Sprint Planning	Sunday 6 PM	1-2 hours	All	Plan upcoming sprint, assign tasks	Video call
Daily Standup	Every day 9 PM	15 min	All	Quick sync: done/doing/blockers	Slack async or quick call
Integration Sync	Wednesday 7 PM	30 min	All	Test integration, resolve dependencies	Video call
Sprint Review	Sunday 3 PM	1 hour	All	Demo progress, retrospective, planning	Video call
Code Review	As needed	30 min	Relevant	Review PRs, architecture decisions	Async or video

Communication Channels

Channel	Purpose	Response Time	Tools
Slack/Discord	Daily communication, quick questions	< 4 hours	Slack workspace
GitHub	Code, issues, PRs, documentation	< 24 hours	GitHub repo
WhatsApp/iMessage	Urgent issues, meeting coordination	< 1 hour	Phone
Zoom/Meet	Video meetings, pair programming	Scheduled	Google Meet
Google Drive	Shared docs, reports, presentations	N/A	Google Drive folder
Email	Formal communication with instructor	< 24 hours	University email

Communication Protocols

Smart Parking Management System - Project Documentation

Slack Channels:

- `#general` - General discussion, announcements
- `#development` - Technical discussions, code questions
- `#integration` - Integration issues and testing
- `#documentation` - Documentation collaboration
- `#standup` - Daily status updates

Daily Standup Format (Async on Slack):

Yesterday:
- [What I completed]

Today:
- [What I'm working on]

Blockers:

- [Any issues preventing progress]

GitHub Workflow:

- Create feature branch from main: `feature/your-feature-name`
- Make commits with clear messages: `feat: add spatial analysis pipeline`
- Create PR when feature is complete
- Request review from relevant team member
- Merge after 1 approval (or immediately if urgent)

Status Reporting

Weekly Status Report (Sunday Sprint Review):

Sprint #: [Number]
Week: [Date Range]

Completed:
- [Achievement 1]
- [Achievement 2]

🚧 In Progress:
- [Task 1] - [Owner] - [% Complete]

Smart Parking Management System - Project Documentation

- [Task 2] - [Owner] - [% Complete]

📅 Next Sprint:
- [Planned task 1]
- [Planned task 2]

🚨 Risks/Blockers:
- [Issue 1] - [Severity] - [Mitigation]

📊 Metrics:
- Model mAP: [X%]
- Latency: [X] seconds
- Test coverage: [X%]
- Sprint velocity: [X] hours

Escalation Path

Level 1 - Team Resolution (Most issues)

- Discuss in daily standup or Slack
- Resolve within team through collaboration
- Time to resolve: Same day

Level 2 - Team Meeting (Blocking issues)

- Schedule emergency team meeting
- Brainstorm solutions together
- Assign action items with owners
- Time to resolve: Within 24 hours

Level 3 - Instructor Consultation (Major blockers)

- Document the issue and attempts to resolve
- Schedule office hours or email instructor
- Get external guidance
- Time to resolve: Within 48 hours

11. QUALITY MANAGEMENT

Smart Parking Management System - Project Documentation

Quality Standards

Code Quality

- **Python:** PEP 8 compliant, type hints for functions
- **JavaScript:** ESLint configuration, consistent style
- **Documentation:** Inline comments for complex logic
- **Naming:** Descriptive variable/function names
- **Modularity:** Functions <50 lines, clear separation of concerns

Testing Requirements

- **Unit Tests:** Critical functions have tests (goal: 60% coverage)
- **Integration Tests:** Component interfaces tested weekly
- **System Tests:** End-to-end workflow tested in Sprint 5
- **User Testing:** Team members test UI workflows

Performance Standards

- **Model Accuracy:** mAP@0.5 >85%
- **Inference Speed:** >10 FPS on Raspberry Pi (15 FPS goal)
- **Latency:** <3 seconds end-to-end (5 seconds acceptable)
- **Uptime:** >90% during final week testing
- **Data Integrity:** 100% (no data loss)

Code Review Process

Every PR Must Have:

1. Clear description of changes
2. Tests added/updated (if applicable)
3. Documentation updated (if applicable)
4. No linting errors
5. Approval from 1 team member

Review Checklist:

- Code is readable and well-commented
- No obvious bugs or logic errors
- Follows project coding standards
- No security vulnerabilities
- Performance considerations addressed

Smart Parking Management System - Project Documentation

- Tests pass (if applicable)

Timeline:

- Small PRs (<100 lines): Review within 12 hours
- Large PRs (>100 lines): Review within 24 hours
- Critical/blocking PRs: Review immediately

Testing Strategy

Test Type	Week	Responsibility	Tools
Unit Tests	2-5	Developer	pytest, jest
Integration Tests	3-5	All	Manual + scripts
Performance Tests	4-5	Naimul, Armaan	Custom benchmarks
User Acceptance	5	Honor	Manual testing
Regression Tests	4-5	All	Automated suite

Definition of Done

A task is **DONE** when:

- Code written and tested locally
- Follows coding standards (linting passes)
- Unit tests written and passing (if applicable)
- Integrated with dependent components
- Code reviewed and approved (for PRs)
- Merged to appropriate branch
- Documentation updated (README, comments, docs)
- Verified in staging/test environment
- Accepted by task owner

Smart Parking Management System - Project Documentation

12. CHANGE MANAGEMENT

Change Control Process

Given the tight 5-week timeline, change management must be **extremely strict**.

Minor Changes (<2 hours, no dependencies)

- Discuss in daily standup
- Implement immediately if agreed
- Update task tracker

Major Changes (>2 hours, affects timeline/scope)

- **DO NOT IMPLEMENT** without team approval
- Submit change request in Slack #general
- Emergency team meeting (30 min max)
- Decision: Approve, Defer to post-project, or Reject
- If approved: Update project plan and communicate

Change Request Template

🚨 CHANGE REQUEST

Requested by: [Name]
Date: [Date]
Sprint: [Number]
Priority: 🔴 High / 🟡 Medium / 🟢 Low

DESCRIPTION:
[What needs to change]

JUSTIFICATION:
[Why this is necessary]

IMPACT ANALYSIS:
- Time required: [X hours/days]
- Affects: [Team member(s)]
- Dependencies: [What else is impacted]
- Risk: [What could go wrong]

Smart Parking Management System - Project Documentation

ALTERNATIVES CONSIDERED:
1. [Option 1]
2. [Option 2]

RECOMMENDATION:
[Approve/Defer/Reject with reasoning]

DECISION:
Status: [Pending/Approved/Deferred/Rejected]
Decided by: [Team]
Date: [Date]

Notes: [Any conditions]

Scope Protection Rules

Automatic REJECTION criteria:

- Any feature not in original scope
- Any change requiring >4 hours in Weeks 1-3
- Any change requiring >2 hours in Weeks 4-5
- Any "nice to have" after Sprint 3

"Future Work" Parking Lot:

- Keep a document of deferred ideas
- Mention in final presentation
- Consider for post-project enhancement

13. BUDGET & COST MANAGEMENT

Project Budget Breakdown

Category	Item	Est. Cost	Actual	Variance	Owner
Hardware	Raspberry Pi 4 (4GB)	$55	TBD	-	Naimul

Smart Parking Management System - Project Documentation

	OAK-D Camera	$150	TBD	-	Naimul
	MicroSD 32GB	$10	TBD	-	Naimul
	Power Supply	$10	TBD	-	Naimul
	Cables & Accessories	$5	TBD	-	Naimul
Cloud	AWS (beyond free tier)	$15	TBD	-	Armaan
	Domain (optional)	$0-12	TBD	-	Honor
Misc	Contingency (10%)	$25	TBD	-	All
TOTAL		**$270-282**	**TBD**	**TBD**	

Cost Control Measures

AWS Cost Management:

- Set billing alerts at $5, $10, $15
- Monitor usage daily in Sprint 3-5
- Use AWS Free Tier calculator
- Shut down unused resources nightly
- Delete test data regularly

Expected AWS Usage (Free Tier Limits):

- Lambda: 1M requests/month (sufficient)
- DynamoDB: 25 GB storage (sufficient)
- API Gateway: 1M requests/month (sufficient)
- Data transfer: 100 GB/month (monitor)

If Free Tier Exceeded:

- Optimize to reduce requests
- Reduce data retention
- Use caching aggressively

Smart Parking Management System - Project Documentation

- Team splits overage cost

Funding & Reimbursement

Funding Sources:

1. Team personal funds (split 3 ways)
2. School lab equipment (if available - check with instructor)
3. AWS Educate credits (if enrolled)

Cost Sharing Agreement:

- Hardware: Team decides (keep/sell/donate after project)
- Cloud: Split any overages equally
- Misc: Individual responsibility

Purchase Timeline:

- **By Nov 4:** Purchase/procure all hardware
- **By Nov 6:** Hardware received and tested

14. SUCCESS METRICS & KPIs

Key Performance Indicators

KPI	Target	Measurement	Frequency	Owner
Model Accuracy (mAP@0.5)	>85%	Validation set evaluation	Week 2	Naimul
Inference FPS	>10 (goal: 15)	Benchmark on Pi	Weeks 2-5	Naimul
End-to-End Latency	<3 sec (accept: <5)	Timed measurements	Weeks 3-5	All
Dataset Size	100+ images	Count in repo	Week 1	Armaan

Smart Parking Management System - Project Documentation

API Response Time	<500ms	CloudWatch metrics	Weeks 3-5	Arma an
Frontend Load Time	<2 seconds	Lighthouse score	Weeks 4-5	Honor
Test Coverage	>60%	pytest-cov report	Weeks 3-5	All
Sprint Completion	100%	Task tracking	Weekly	All
Budget Adherence	<$300	Expense tracking	Continuou s	All
Documentation Quality	Complete & clear	Peer review	Week 5	Honor

Sprint Velocity Tracking

Measure Each Sprint:

- Planned tasks vs completed tasks
- Estimated hours vs actual hours
- Blockers encountered and resolution time
- Code commits per team member
- PR merge rate

Sprint Velocity Goals:

- Sprint 1-2: Establish baseline
- Sprint 3-4: Maintain or improve
- Sprint 5: Focus on quality over quantity

Project Success Criteria

Technical Success (Must Have):

- System detects parking occupancy with >80% accuracy (85% goal)
- Latency under 5 seconds (3 seconds goal)
- All 5 sprint milestones completed
- Live demo executes without critical failures
- Code committed to GitHub and documented

Smart Parking Management System - Project Documentation

Feature Success (Must Have):

- Real-time parking detection working
- Nearest spot algorithm functional
- Cloud infrastructure operational
- Frontend displays live data
- Billing system calculates correctly

Documentation Success (Must Have):

- Technical architecture documented
- User manual completed
- Deployment guide available
- API documentation clear
- Presentation delivered

Team Success (Nice to Have):

- Equitable contribution from all members
- No major conflicts or delays
- Positive collaboration experience
- Learning objectives met

15. ASSUMPTIONS & CONSTRAINTS

Assumptions

1. **Hardware:** OAK-D camera and Raspberry Pi function as expected
2. **Access:** School provides access to parking lot for data collection
3. **Compute:** Team has access to GPU for model training (or accepts longer training times)
4. **Cloud:** AWS Free Tier sufficient for project scope
5. **Network:** Reliable internet available for edge device and demo
6. **Time:** Team can commit 20 hours/week per person
7. **Support:** Instructor available for technical guidance
8. **Environment:** Parking lot has stable lighting (daytime testing)
9. **Skills:** Team has baseline Python, web development knowledge
10. **Collaboration:** All team members available throughout 5 weeks

Smart Parking Management System - Project Documentation

Constraints

Time Constraints:

- ⏰ **HARD DEADLINE: December 8, 2024**
- Only 5 weeks (35 days) total
- Thanksgiving holiday Week 4 (potential disruption)
- Final exam period approaching
- No extensions possible

Resource Constraints:

- 3-person team (limited bandwidth)
- ~$300 budget maximum
- Single Raspberry Pi device
- Single OAK-D camera
- AWS Free Tier limits
- No dedicated QA or DevOps support

Technical Constraints:

- Raspberry Pi CPU/RAM limitations
- OAK-D camera range (~15 meters)
- YOLOv8 Nano model complexity
- Real-time processing requirement
- Network bandwidth at deployment location
- Browser compatibility requirements

Scope Constraints:

- Single parking lot location only
- No mobile applications
- No payment processing integration
- Maximum 20-30 parking spaces
- Basic UI only (not production-grade)
- Limited error recovery
- No multi-user authentication

Knowledge Constraints:

- Limited experience with OAK-D
- First time deploying edge ML models
- Learning AWS while building
- Limited computer vision expertise

Smart Parking Management System - Project Documentation

Constraint Management

Time Pressure Strategies:

- Work in parallel wherever possible
- Ruthless prioritization of MVP features
- Accept "working" over "perfect"
- Use proven libraries/frameworks
- Minimize custom code

Resource Optimization:

- Reuse open-source components
- Use free tier services maximally
- Share knowledge through pair programming
- Cross-train to reduce single points of failure

16. DEPENDENCIES

External Dependencies

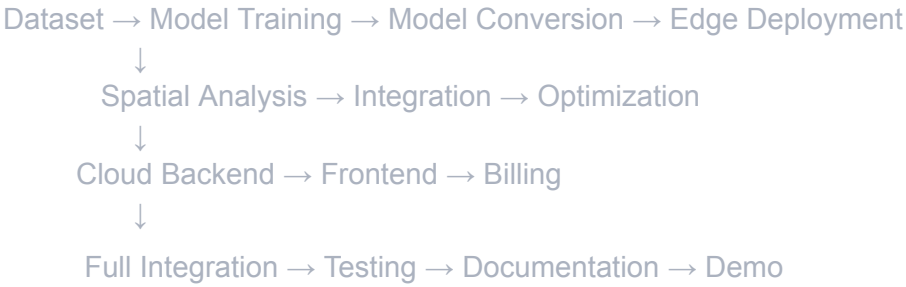
Dependency	Type	Impact if Unavailable	Mitigation	Owner
OAK-D Camera	Hardware	Critical - cannot do depth analysis	Use laptop webcam + simulated depth	Naimul
Raspberry Pi	Hardware	Critical - cannot deploy edge	Use laptop as edge device	Naimul
AWS Free Tier	Cloud	High - need alternate hosting	Use Heroku/Railway for backend	Armaan
Parking Lot Access	Data	High - cannot collect custom dataset	Use public parking datasets (PKLot)	Armaan
YOLOv8 Library	Software	Medium - need alternate model	Use YOLOv5 or MobileNet-SSD	Naimul

Smart Parking Management System - Project Documentation

GitHub	Collaboration	Medium - need alternate	GitLab or local Git server	All
Internet Connectivity	Infrastructure	High - cannot demo live	Pre-record demo video	All

Internal Dependencies

Critical Path:



Dependency Chain:

1. **Week 1:** Dataset collection → Model training can begin
2. **Week 2:** Trained model → Edge deployment can begin
3. **Week 2:** Spatial pipeline → Integration can begin
4. **Week 3:** Edge working → Cloud can receive data
5. **Week 3:** Cloud working → Frontend can display data
6. **Week 4:** Full integration → Optimization can begin
7. **Week 5:** System complete → Documentation can finalize

Dependency Management

Risk Mitigation:

- **Parallel Work:** Start frontend with mock data before cloud ready
- **Early Integration:** Test integration weekly, not just at end
- **Fallback Plans:** Have Plan B for every critical dependency
- **Communication:** Daily standup to identify blocking dependencies
- **Buffer Time:** Sprint 5 entirely dedicated to integration/fixes

Dependency Tracking:

Smart Parking Management System - Project Documentation

- Mark tasks in GitHub with dependencies
- Discuss in Sprint Planning
- Highlight in daily standups
- Escalate blocking dependencies immediately

17. LESSONS LEARNED & CONTINUOUS IMPROVEMENT

Learning Objectives

Technical Skills:

- Computer vision and object detection (YOLOv8)
- Edge computing and IoT deployment
- Cloud architecture (AWS)
- Full-stack web development
- Spatial analysis and depth perception
- Real-time data processing
- API design and integration

Soft Skills:

- Agile/Scrum project management
- Team collaboration under tight deadlines
- Technical documentation writing
- Presentation and demo skills
- Problem-solving under pressure
- Time management and prioritization

Weekly Retrospectives

Every Sunday Sprint Review includes 30-min Retrospective:

Format:

1. **What Went Well** (Continue)
2. **What Didn't Go Well** (Stop)
3. **What to Improve** (Start)
4. **Action Items** (Who, What, When)

Smart Parking Management System - Project Documentation

Retrospective Questions:

- Did we meet our sprint goal?
- What surprised us this week?
- What slowed us down?
- What should we celebrate?
- What

NH

Continue

will we change next sprint?

Knowledge Capture

Throughout Project:

- Document technical decisions in GitHub wiki
- Create troubleshooting guides as issues arise
- Record "gotchas" and solutions
- Take screenshots/videos of milestones
- Save error messages and fixes

End of Project:

- Comprehensive README with lessons learned
- "If we did this again" document
- Technical debt and future work list
- Recommendations for future teams
- Archive all artifacts

18. PROJECT CLOSURE CHECKLIST

Technical Closure (Due: Dec 7)

Code & Deployment:

- All code committed to main branch
- GitHub repo organized with clear README
- All branches merged or closed

Smart Parking Management System - Project Documentation

- .blob model file in repository
- Requirements.txt / package.json complete
- Environment setup instructions documented

System Verification:

- System fully functional end-to-end
- All tests passing
- Performance metrics documented
- Known issues logged with workarounds
- Demo environment tested and ready

Documentation Closure (Due: Dec 7)

Required Documents:

- **Spatial Analysis Report** (10-15 pages)
  - Methodology
  - Algorithms
  - Results
  - Accuracy validation
- **Cloud Infrastructure Report** (10-15 pages)
  - Architecture
  - API documentation
  - Data flow
  - Billing system
- **User Manual** (5-10 pages)
  - Installation guide
  - User workflows
  - Troubleshooting
  - FAQs
- **Deployment Guide** (5-10 pages)
  - Hardware setup
  - Software configuration
  - Cloud deployment
  - Testing procedures
- **API Documentation**
  - Endpoints
  - Request/response formats
  - Authentication
  - Examples

Smart Parking Management System - Project Documentation

- **README.md** (Comprehensive)
    - Project overview
    - Setup instructions
    - Usage examples
    - Team member contributions

Presentation Closure (Due: Dec 7)

Presentation Materials:

- Slide deck (25-30 slides)
- Speaker notes for each section
- Demo script with timing
- Backup demo video (5 min)
- Q&A preparation document
- Handout/one-pager (optional)

Rehearsal Checklist:

- 3 full rehearsals completed
- Timing perfected (30 min total)
- Transitions smooth
- Demo tested 5+ times
- Backup plan ready
- All equipment tested

Administrative Closure (Due: Dec 8)

Submission:

- All deliverables uploaded to course portal
- GitHub repository made public (or shared with instructor)
- Presentation submitted
- Demo video uploaded
- Final report PDF submitted

Team:

- All expenses reconciled
- Equipment returned (if borrowed)

Smart Parking Management System - Project Documentation

- Thank you notes to instructor/TA
- Team celebration planned 🎉

Handoff:

- Archive project in Google Drive
- Save all credentials securely
- Document AWS resources for cleanup
- Share lessons learned with peers

19. APPENDICES

Appendix A: Glossary

Term	Definition
YOLOv8	You Only Look Once v8, latest real-time object detection model
OAK-D	OpenCV AI Kit with Depth, Intel Myriad X powered stereo camera
Raspberry Pi	Single-board computer for edge computing
mAP	Mean Average Precision, metric for object detection accuracy
Blob	Binary format for deploying models on Intel Myriad X processors
Edge Computing	Processing data locally on device rather than in cloud
Spatial Analysis	Converting 2D image coordinates to 3D real-world positions
DynamoDB	AWS NoSQL database service
Lambda	AWS serverless compute service
API Gateway	AWS service for creating/managing APIs
MQTT	Lightweight pub-sub messaging protocol for IoT
Latency	Time delay from detection to display
FPS	Frames Per Second, measure of processing speed
Sprint	One-week iteration in Agile methodology

Smart Parking Management System - Project Documentation

Appendix B: Technical Stack

Hardware:

- Raspberry Pi 4 (4GB RAM)
- OAK-D Camera (Intel Myriad X)
- MicroSD Card (32GB, Class 10)

Edge Software:

- Raspberry Pi OS (64-bit)
- Python 3.8+
- YOLOv8 (Ultralytics)
- DepthAI SDK
- OpenCV
- NumPy, SciPy

Cloud Infrastructure:

- AWS Lambda (Python 3.9 runtime)
- AWS DynamoDB
- AWS API Gateway
- AWS CloudWatch (monitoring)

Frontend:

- React.js or Vue.js
- HTML5, CSS3, JavaScript ES6+
- Chart.js or D3.js (visualizations)
- Axios (API calls)
- TailwindCSS or Bootstrap (styling)

Development Tools:

- Git & GitHub
- VS Code
- Postman (API testing)
- AWS CLI
- Slack/Discord

Data Tools:

- Roboflow or LabelImg (annotation)
- OpenVINO (model conversion)

Smart Parking Management System - Project Documentation

- Blobconverter

Appendix C: Useful Resources

Documentation:

- YOLOv8: https://docs.ultralytics.com
- DepthAI: https://docs.luxonis.com
- AWS Lambda: https://docs.aws.amazon.com/lambda/
- DynamoDB: https://docs.aws.amazon.com/dynamodb/
- React: https://react.dev

Tutorials:

- YOLOv8 Training: https://blog.roboflow.com/how-to-train-yolov8/
- OAK-D Setup: https://docs.luxonis.com/projects/api/en/latest/
- AWS Serverless: https://aws.amazon.com/getting-started/

Datasets (if needed):

- PKLot: http://web.inf.ufpr.br/vri/databases/parking-lot-database/
- CNRPark: http://cnrpark.it/

Appendix D: Contact Information

Name	Role	Email	Phone	Slack
Naimul	ML/Hardware Lead	naimul@[domain]	[###-###-####]	@naimul
Armaan	Data/Cloud Lead	armaan@[domain]	[###-###-####]	@armaan
Honor Byansi	Frontend/Docs Lead	honor@[domain]	[###-###-####]	@honor
Instructor	Project Sponsor	instructor@[domain]	[###-###-####]	N/A

Team Availability:

- **Best contact hours:** 9 AM - 10 PM EST
- **Emergency contact:** WhatsApp group

Smart Parking Management System - Project Documentation

- **Response commitment:** Within 4 hours during waking hours

Appendix E: Meeting Schedule

Recurring Meetings (5 Weeks):

Day	Time	Meeting	Duration	Location
Monday-Saturday	9:00 PM	Daily Standup	15 min	Slack (async)
Wednesday	7:00 PM	Integration Sync	30 min	Zoom/Meet
Sunday	3:00 PM	Sprint Review	1 hour	Zoom/Meet
Sunday	4:00 PM	Sprint Retrospective	30 min	Zoom/Meet
Sunday	6:00 PM	Sprint Planning	1-2 hours	Zoom/Meet

Ad-hoc Meetings:

- Code review sessions (as needed)
- Pair programming (as needed)
- Emergency team meetings (within 4 hours of request)

SIGN-OFF & COMMITMENT

This project charter represents our team's commitment to delivering a high-quality Smart Parking Management System by December 8, 2024.

We commit to:

- 20 hours/week individual contribution
- Daily communication and transparency
- Supporting each other through challenges

Smart Parking Management System - Project Documentation

- Delivering on our sprint commitments
- Meeting quality standards
- Completing all documentation
- Presenting professionally on December 8

Team Member Signatures:

Naimul (ML/Hardware Lead)
Signature: _____ Date: _____
Role: Model training, hardware setup, edge deployment

Armaan (Data/Cloud Lead)
Signature: _____ Date: _____
Role: Dataset creation, AWS infrastructure, API development

Honor Byansi (Frontend/Documentation Lead)
Signature: _____ Date: _____
Role: UI/UX design, web application, technical documentation

Project Sponsor Approval:

Noopa Jagadeesh
Signature: _____ Date: _____
Title: [Course Instructor/Project Sponsor]

Document Information:

- **Version:** 1.0
- **Created:** November 4, 2024
- **Last Updated:** November 4, 2024
- **Next Review:** November 11, 2024 (Sprint 1 Review)
- **Final Review:** December 8, 2024 (Project Completion)